

Cruise Critic Database System

EECS3421: Final Project Report

Melhem Fares*
York University
Toronto, Ontario, Canada
mfares@my.yorku.ca

Erfan Razmand*
York University
Toronto, Ontario, Canada
erfanuni@my.yorku.ca

Zuhaib Khan*
York University
Toronto, Ontario, Canada
zuhaib@my.yorku.ca

Johnson Dang*
York University
Toronto, Ontario, Canada
jdang04@my.yorku.ca

Zayed AlMarzooqi*
York University
Toronto, Ontario, Canada
mrzayed@my.yorku.ca

Abstract

This report presents the design and implementation of the Cruise Critic Database System, a relational database project for modeling cruise-search, itinerary, pricing, review, and ship-information features. The system is based on the Cruise Critic travel-planning platform and is designed to support core application functions such as cruise search, cruise comparison, user reviews, bonus offers, partner deals, departure dates, and management reporting. The project includes a conceptual model, business rules, a logical relational schema, SQL table definitions, sample data, integrity constraints, and analytical queries. The completed design demonstrates how normalized relational tables, primary keys, foreign keys, bridge tables, and SQL joins can support a structured cruise-planning database.

Keywords

relational database, SQL, database design, Cruise Critic, ER modeling, normalization, data integrity, analytical queries

1 Introduction

Cruise Critic is an online cruise travel platform that helps travelers research cruises, compare ships, examine itineraries, review deals, and read user feedback. Cruise planning requires combining many types of information, including cruise lines, ships, ports, destinations, departure dates, prices, bonus offers, reviews, and ratings. Because these data categories are connected, a relational database is an appropriate foundation for organizing and retrieving them.

This project designs and implements a relational database system for a Cruise Critic-style application. The system stores structured cruise data and supports common user and management tasks. Users should be able to search cruises by destination, departure port, travel dates, cruise line, duration, and ship. They should also be able to review cruise details, compare deals, and read rating information. From a management perspective, the database should support reporting queries such as identifying highly rated cruises, cruises departing from a specific port, cruises with multiple bonus offers, and cruises using ships with high passenger capacity.

The report is organized as follows. Section 2 introduces the background, objectives, and functional requirements. Section 3 describes the database model, business rules, and logical relational

design. Section 4 presents the implementation, table design, sample data, and SQL queries. Section 5 evaluates the project, discusses limitations, and summarizes lessons learned.

2 Background and Objectives

2.1 Organization Background

Cruise Critic is a well-known cruise travel platform that provides cruise reviews, cruise deals, ship information, destination details, itineraries, and travel-planning resources. It serves a wide range of users, including first-time travelers, experienced cruise passengers, travel researchers, and industry partners.

The platform is useful because cruise booking is complex. Travelers often need to compare options across departure ports, dates, cruise lines, ship features, onboard amenities, destinations, prices, and reviews. A database system supporting this type of platform must therefore store data across many related entities while preserving consistency and searchability.

2.2 Business Objective

The main business objective is to reduce uncertainty in cruise planning by centralizing credible and structured travel information. A Cruise Critic-style database should help users find relevant cruises, compare available options, evaluate pricing and deals, and read user-generated reviews.

For business stakeholders, the database can support management reporting and operational analysis. For example, the platform can identify which cruises have strong ratings, which departure ports are most relevant, which cruises have bonus offers, and which ships meet passenger-capacity requirements.

2.3 Application Scope

The scope of this project is the relational database layer. The project does not implement a full production website, login system, or front-end interface. Instead, it focuses on the database structures needed to support the major application functions.

The system stores cruise records, ships, cruise lines, departure dates, ports, itineraries, bonus offers, deals, price tiers, customers, reviews, review categories, and ratings. These entities are normalized into separate tables and connected using primary keys, foreign keys, and bridge tables.

*All authors contributed to the project report.

3 Functional Requirements

The main functional requirements are summarized in Table 1.

These requirements guide the database design. For example, searching by departure date requires a departure-date table and a relationship between cruises and dates. Supporting reviews and category-specific ratings requires separate review, review-type, and rating tables.

4 Database Modeling

4.1 Business Rules and Assumptions

The following rules define how the entities behave in the system:

- (1) The community forum is outside the project scope.
- (2) Each cruise has one destination, and each destination may belong to zero or more cruises.
- (3) Each cruise has one departure port, and each departure port may belong to zero or more cruises.
- (4) Each cruise may have many departure dates, and each departure date may belong to many cruises.
- (5) Each cruise has one ship, and each ship may belong to many cruises.
- (6) Each cruise has zero or more bonus offers, and each bonus offer may belong to zero or more cruises.
- (7) Each cruise has one itinerary, and each itinerary belongs to one cruise.
- (8) Each itinerary has one or more ports, and each port may be part of zero or more itineraries.
- (9) Each cruise has one or more deals, and each deal belongs to one cruise.
- (10) Each deal has one or more price tiers, and a price tier may belong to one or more deals.
- (11) Each cruise has zero or more reviews, and each review belongs to one cruise.
- (12) Each review may contain one or more review-category ratings, and each review category may appear in many reviews.
- (13) A cruise is defined by its ship, itinerary, duration, dates, and related cruise attributes.
- (14) Each ship has a launch year, crew count, passenger capacity, and number of shore excursions.
- (15) Each deal includes a description and a partner-site link.

These assumptions lead to a design that uses bridge tables for many-to-many relationships such as cruise-date, cruise-offer, deal-tier, itinerary-port, and review-rating relationships.

4.2 Core Entities

Table 2 summarizes the main entities in the system.

4.3 Logical Relational Design

The final logical design uses CRUISE as the central table. Cruises connect to ships, cruise lines, departure dates, deals, bonus offers, itineraries, reviews, and ratings. The model separates repeating or reusable information into independent tables. This improves normalization and prevents duplicate storage of values such as port names, price tiers, review categories, and bonus offers.

Table 3 summarizes the main relationships.

5 Database Design and Implementation

5.1 Schema Design

The implemented schema uses primary keys, foreign keys, and not-null constraints to enforce data integrity. The main table definitions are summarized in Table 4.

5.2 Representative SQL Table Definitions

The following listing shows representative table definitions from the implementation. The full schema follows the same structure: each table has a primary key, and relationship tables use foreign keys to preserve referential integrity.

Listing 1: Representative SQL table definitions.

```
CREATE TABLE SHIP (
  SHIP_ID NUMERIC(4,0) NOT NULL,
  SHIP_LAUNCH_YEAR NUMERIC(4,0) NOT NULL,
  CREW_COUNT NUMERIC(4,0) NOT NULL,
  PASSENGER_CAPACITY NUMERIC(4,0) NOT NULL,
  SHORE_EXCURSIONS NUMERIC(5,0) NOT NULL,
  PRIMARY KEY (SHIP_ID)
);

CREATE TABLE CRUISE (
  CRUISE_ID NUMERIC(4,0) NOT NULL,
  CRUISE_NAME VARCHAR(64) NOT NULL,
  CRUISE_LENGTH NUMERIC(2,0) NOT NULL,
  SHIP_ID NUMERIC(4,0) NOT NULL,
  PRIMARY KEY (CRUISE_ID),
  FOREIGN KEY (SHIP_ID) REFERENCES SHIP (SHIP_ID)
  ON DELETE NO ACTION
  ON UPDATE CASCADE
);

CREATE TABLE CRUISE_DATE (
  CRUISE_ID NUMERIC(4,0) NOT NULL,
  DEPARTURE_DATE_ID NUMERIC(4,0) NOT NULL,
  PRIMARY KEY (CRUISE_ID, DEPARTURE_DATE_ID),
  FOREIGN KEY (CRUISE_ID) REFERENCES CRUISE (CRUISE_ID)
  ON DELETE NO ACTION
  ON UPDATE CASCADE,
  FOREIGN KEY (DEPARTURE_DATE_ID)
  REFERENCES DEPARTURE_DATE (DEPARTURE_DATE_ID)
  ON DELETE NO ACTION
  ON UPDATE CASCADE
);
```

Listing 2: Representative bridge tables.

```
CREATE TABLE CRUISE_OFFER (
  CRUISE_ID NUMERIC(4,0) NOT NULL,
  BONUS_OFFER_ID NUMERIC(4,0) NOT NULL,
  PRIMARY KEY (CRUISE_ID, BONUS_OFFER_ID),
  FOREIGN KEY (CRUISE_ID) REFERENCES CRUISE (CRUISE_ID)
  ON DELETE NO ACTION
  ON UPDATE CASCADE,
  FOREIGN KEY (BONUS_OFFER_ID)
  REFERENCES BONUS_OFFER (BONUS_OFFER_ID)
  ON DELETE NO ACTION
  ON UPDATE CASCADE
);

CREATE TABLE RATING (
  REVIEW_ID NUMERIC(4,0) NOT NULL,
  REVIEW_TYPE_ID NUMERIC(4,0) NOT NULL,
  SCORE NUMERIC(2,1) NOT NULL,
  PRIMARY KEY (REVIEW_ID, REVIEW_TYPE_ID),
  FOREIGN KEY (REVIEW_ID) REFERENCES REVIEW (REVIEW_ID)
  ON DELETE NO ACTION
  ON UPDATE CASCADE,
  FOREIGN KEY (REVIEW_TYPE_ID)
  REFERENCES REVIEW_TYPE (REVIEW_TYPE_ID)
  ON DELETE NO ACTION
);
```

Table 1: Functional requirements of the Cruise Critic Database System.

Requirement	Description
Cruise Search	Users should be able to search for cruises by destination, cruise line, departure port, travel date, ship, and duration.
Cruise Comparison	Users should be able to compare cruises using cruise features, pricing, ratings, and available offers.
Filtering and Sorting	Users should be able to filter and sort cruise listings to customize their browsing experience.
User Reviews and Ratings	Registered users should be able to write reviews and assign rating scores to cruises they have taken.
Cruise Detail Lookup	Users should be able to view cruise length, ship information, itinerary ports, departure dates, and related deals.
Offer and Deal Tracking	The system should store bonus offers, partner deals, price tiers, and associated cruise information.
Management Reporting	The system should support SQL reports for average ratings, departure ports, bonus offers, ship capacity, and departure dates.

Table 2: Core entity summary.

Entity	Purpose
SHIP	Stores ship-level data including launch year, crew count, passenger capacity, and shore excursions.
CRUISE	Stores main cruise records including cruise name, length, and associated ship.
CRUISE_LINE	Stores cruise-line names and their relationship to cruises.
DEPARTURE_DATE	Stores available departure dates.
CRUISE_DATE	Bridge table connecting cruises to departure dates.
BONUS_OFFER	Stores possible promotional bonus offers.
CRUISE_OFFER	Bridge table connecting cruises to bonus offers.
DEAL	Stores partner deal descriptions, partner links, and cruise association.
PRICE_TIER	Stores price-tier categories such as premium suite, deluxe cabin, and economy cabin.
DEAL_TIER	Bridge table connecting deals to price tiers and prices.
CUSTOMER	Stores user/customer information.
REVIEW	Stores written cruise reviews linked to customers and cruises.
REVIEW_TYPE	Stores rating categories such as dining, entertainment, service, and value.
RATING	Bridge table connecting reviews to rating categories and scores.
ITINERARY	Stores itinerary records connected to cruises.
PORT	Stores port names and locations.
ITINERARY_PORT	Bridge table connecting itineraries to ports with ordering information.

```
ON UPDATE CASCADE
);
```

5.3 Sample Data

The database was populated with sample data representing ships, cruises, cruise lines, dates, offers, deals, price tiers, customers, reviews, ratings, itineraries, and ports. Table 5 summarizes representative sample records.

6 SQL Queries and Results

The system includes management-style SQL queries that demonstrate how the database supports reporting and analysis.

6.1 Query 1: Cruises with Average Rating of at Least 4

This query finds cruises whose average rating score is at least 4.

Listing 3: Find cruises with average rating score of at least 4.

```
SELECT CRUISE.CRUISE_NAME,
       AVG(RATING.SCORE) AS AVERAGE_RATING
FROM CRUISE
JOIN REVIEW ON CRUISE.CRUISE_ID = REVIEW.CRUISE_ID
JOIN RATING ON REVIEW.REVIEW_ID = RATING.REVIEW_ID
GROUP BY CRUISE.CRUISE_NAME
HAVING AVG(RATING.SCORE) >= 4;
```

6.2 Query 2: Cruises Departing from Toronto

This query identifies cruises whose first itinerary port is Toronto.

Table 3: Relationship summary and relational implementation.

Relationship	Cardinality	Implementation
Ship to Cruise	One-to-many	SHIP_ID appears as a foreign key in CRUISE.
Cruise to Departure Date	Many-to-many	Implemented using CRUISE_DATE.
Cruise to Bonus Offer	Many-to-many	Implemented using CRUISE_OFFER.
Cruise to Deal	One-to-many	CRUISE_ID appears as a foreign key in DEAL.
Deal to Price Tier	Many-to-many	Implemented using DEAL_TIER, which also stores price.
Cruise to Review	One-to-many	CRUISE_ID appears as a foreign key in REVIEW.
Customer to Review	One-to-many	CUSTOMER_ID appears as a foreign key in REVIEW.
Review to Review Type	Many-to-many	Implemented using RATING, which stores category score.
Cruise to Itinerary	One-to-many	CRUISE_ID appears as a foreign key in ITINERARY.
Itinerary to Port	Many-to-many	Implemented using ITINERARY_PORT, which stores port order.

Table 4: Implemented table design summary.

Table	Main Attributes	Key Design
SHIP	SHIP_ID, SHIP_LAUNCH_YEAR, CREW_COUNT, PASSENGER_CAPACITY, SHORE_EXCURSIONS	Primary key on SHIP_ID.
CRUISE	CRUISE_ID, CRUISE_NAME, CRUISE_LENGTH, SHIP_ID	Foreign key references SHIP.
CRUISE_LINE	CRUISE_LINE_ID, CRUISE_LINE_NAME, CRUISE_ID	Foreign key references CRUISE.
DEPARTURE_DATE	DEPARTURE_DATE_ID, DEPART_DATE	Primary key on date identifier.
CRUISE_DATE	CRUISE_ID, DEPARTURE_DATE_ID	Composite primary key and two foreign keys.
BONUS_OFFER	BONUS_OFFER_ID, BONUS_OFFER_NAME	Primary key on offer identifier.
CRUISE_OFFER	CRUISE_ID, BONUS_OFFER_ID	Composite primary key and two foreign keys.
DEAL	DEAL_ID, DEAL_DESCRIPTION, PARTNER_SITE, CRUISE_ID	Foreign key references CRUISE.
PRICE_TIER	PRICE_TIER_ID, PRICE_TIER_NAME	Primary key on tier identifier.
DEAL_TIER	PRICE_TIER_ID, DEAL_ID, PRICE	Composite key with price attribute.
CUSTOMER	CUSTOMER_ID, FIRST_NAME, LAST_NAME, EMAIL	Primary key on customer identifier.
REVIEW	REVIEW_ID, REVIEW_DESCRIPTION, CRUISE_ID, CUSTOMER_ID	Foreign keys to cruise and customer.
REVIEW_TYPE	REVIEW_TYPE_ID, REVIEW_TYPE_NAME	Primary key on review type.
RATING	REVIEW_ID, REVIEW_TYPE_ID, SCORE	Composite primary key.
ITINERARY	ITINERARY_ID, CRUISE_ID	Foreign key references CRUISE.
PORT	PORT_ID, PORT_NAME, PORT_LOCATION	Primary key on port identifier.
ITINERARY_PORT	ITINERARY_ID, PORT_ID, ORDERING	Composite key with ordering attribute.

Listing 4: Find cruises whose departure port is Toronto.

```

SELECT CRUISE.CRUISE_NAME,
       PORT.PORT_NAME AS DEPARTURE_PORT
FROM CRUISE
JOIN ITINERARY
  ON ITINERARY.CRUISE_ID = CRUISE.CRUISE_ID
JOIN ITINERARY_PORT
  ON ITINERARY.ITINERARY_ID = ITINERARY_PORT.ITINERARY_ID
JOIN PORT
  ON PORT.PORT_ID = ITINERARY_PORT.PORT_ID
WHERE PORT.PORT_NAME LIKE '%Toronto'
AND ITINERARY_PORT.ORDERING = 1;

```

6.3 Query 3: Cruises with Two or More Bonus Offers

This query finds cruises that have at least two bonus offers.

Listing 5: Find cruises with two or more bonus offers.

```

SELECT CRUISE.CRUISE_NAME,
       COUNT(BONUS_OFFER.BONUS_OFFER_ID) AS BONUS_OFFER_COUNT
FROM CRUISE
JOIN CRUISE_OFFER
  ON CRUISE_OFFER.CRUISE_ID = CRUISE.CRUISE_ID
JOIN BONUS_OFFER
  ON BONUS_OFFER.BONUS_OFFER_ID = CRUISE_OFFER.BONUS_OFFER_ID
GROUP BY CRUISE.CRUISE_NAME
HAVING COUNT(BONUS_OFFER.BONUS_OFFER_ID) >= 2;

```

6.4 Query 4: Large-Capacity Ships Departing After November 1, 2025

This query finds cruises with ships that can hold at least 3000 passengers and depart after November 1, 2025.

Table 5: Representative sample data used in the database.

Table	Example Records
SHIP	Ship IDs 201–203 with launch years 2019–2021, crew counts 750–900, passenger capacities 2800–3500, and shore excursions 12–18.
CRUISE	Caribbean Adventure, Mediterranean Escape, Alaskan Explorer, and Baltic Discovery.
CRUISE_LINE	Carnival Cruise Line, Norwegian Cruise Line, Princess Cruises, and Royal Caribbean.
DEPARTURE_DATE	Dates such as 2025-12-15, 2025-12-22, 2025-11-10, 2025-10-05, and 2025-09-25.
BONUS_OFFER	Free Spa Access, Complimentary Wine, Onboard Shopping Voucher, and Free WiFi Package.
PRICE_TIER	Premium Suite, Deluxe Cabin, and Economy Cabin.
CUSTOMER	Sample customers such as John Doe, Alice Smith, Robert Brown, and Maria Garcia.
REVIEW_TYPE	Dining/Food, Entertainment, Value for your money, and Service.
PORT	Port of Toronto, Port of Montreal, Port of Philadelphia, Port of Baltimore, and Port of New York.

Table 6: Query 1 result.

Cruise Name	Average Rating
Caribbean Adventure	4.5
Mediterranean Escape	4.0

Table 7: Query 2 result.

Cruise Name	Departure Port
Caribbean Adventure	Port of Toronto

Table 8: Query 3 result.

Cruise Name	Bonus Offer Count
Baltic Discovery	2

Listing 6: Find cruises on ships with at least 3000 passengers departing after November 1, 2025.

```
SELECT CRUISE.CRUISE_NAME,
       SHIP.PASSENGER_CAPACITY,
       DEPARTURE_DATE.DEPART_DATE
FROM CRUISE
JOIN SHIP
  ON SHIP.SHIP_ID = CRUISE.SHIP_ID
JOIN CRUISE_DATE
  ON CRUISE_DATE.CRUISE_ID = CRUISE.CRUISE_ID
JOIN DEPARTURE_DATE
  ON DEPARTURE_DATE.DEPARTURE_DATE_ID =
    CRUISE_DATE.DEPARTURE_DATE_ID
WHERE SHIP.PASSENGER_CAPACITY >= 3000
AND DEPARTURE_DATE.DEPART_DATE > '2025-11-01';
```

Table 9: Query 4 result.

Cruise Name	Capacity	Departure Date
Caribbean Adventure	3000	2025-12-15
Caribbean Adventure	3000	2025-12-22

7 Evaluation

7.1 Project Achievement

The project successfully produced a normalized relational database design for a Cruise Critic-style cruise-planning platform. The final schema supports major application features including cruise search, cruise details, ship information, departure dates, itineraries, pricing, bonus offers, customer reviews, and rating categories.

The implementation demonstrates several key database concepts. Primary keys uniquely identify records, foreign keys preserve referential integrity, and bridge tables support many-to-many relationships. The SQL queries show that the database can answer useful management questions involving joins, grouping, aggregation, filtering, and conditional logic.

7.2 Advantages of the Database Design

The main advantage of the design is normalization. Instead of storing repeated information directly inside cruise records, the model separates ships, ports, offers, deals, price tiers, reviews, and rating categories into independent tables. This reduces redundancy and makes updates easier.

The design also supports flexible querying. For example, one query can join cruises, reviews, and ratings to calculate average scores, while another can join cruises, itineraries, itinerary ports, and ports to find departures from Toronto. This confirms that the schema supports both application use cases and management-style reporting.

7.3 Limitations

The project has several limitations. First, the sample dataset is small, so it demonstrates functionality but does not represent the full scale of a real cruise platform. Second, some website features, such as forums and full user-account authentication, are outside the project scope. Third, the design does not include advanced constraints such as unique email validation, rating-score range checks, or full date validation beyond the basic table definitions. These would be useful in a production-ready database.

The project was also implemented using a database environment inside Visual Studio Code. This made SQL development convenient, but it was not ideal for real-time collaboration because multiple

team members could not easily work in the same database console at once.

7.4 Future Improvements

Future work could expand the database in several ways. The schema could include user authentication, forum posts, destination-region tables, booking history, cabin categories, availability tracking, and more detailed price-history data. The SQL queries could also be expanded to support trend analysis, top-rated cruise lines, most popular ports, price comparisons by tier, and customer-review summaries.

A larger dataset would also allow stronger testing of query performance. Indexes could be added on common search fields such as cruise name, cruise line, departure date, port name, and review score. In a production system, query optimization would be important because cruise-search platforms may contain thousands of ships, routes, reviews, and deals.

8 Team Contributions

Table 10 summarizes the approximate contribution areas documented for the project.

9 Conclusion

This project designed and implemented a relational database system for a Cruise Critic-style cruise-planning platform. The database models cruises, ships, cruise lines, departure dates, itineraries, ports, deals, price tiers, bonus offers, customers, reviews, review types, and ratings. Through normalized tables and referential integrity constraints, the design supports reliable storage and retrieval of cruise-related information.

The SQL queries demonstrate that the database can answer practical questions about average ratings, departure ports, bonus offers, ship capacity, and departure dates. Overall, the project provided experience with database modeling, business-rule definition, ER design, schema implementation, sample-data insertion, and analytical SQL query design.

References

- (1) Cruise Critic. Cruise Reviews, Cruise Deals and Cruises. <https://www.cruisecritic.com/>
- (2) Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson.
- (3) Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill.

Table 10: Team contribution summary.

Member	Main Contribution	Time
Melhem Fares	Worked on business rules, finalized the ER model, relational schema model, and database tables.	5.0 hours
Erfan Razmand	Completed the proposal and contributed to the introduction and preliminary-study sections of the report.	2.5 hours
Zuhaib Khan	Worked on business rules and the first draft of the ER model.	2.5 hours
Johnson Dang	Coordinated group communication, edited grammar and spelling, wrote evaluation content, helped finalize the introduction, and improved report formatting.	6.5 hours
Zayed AlMarzooqi	Worked on SQL queries based on the tables and helped troubleshoot database-related questions.	2.5 hours